

Chapter 17 : Amortized Analysis

1) In an amortized analysis, the time required to perform a sequence of data-structure operations is averaged over all the operations performed. Amortized analysis can be used to show that the average cost of an operation is small, if one averages over a sequence of operations, even though a single operation in the sequence might be expensive.

Amortized analysis differs from average case analysis in that probability is not involved. An amortized analysis guarantees the average performance of each operation in the worst case.

Three most common techniques used in amortized analysis are aggregate analysis, accounting method and potential method.

2) Aggregate analysis :

In aggregate analysis, we show that a sequence of n operations takes a "worst-case time" of $T(n)$ in total so that the average cost or amortized cost per operation is $T(n)/n$. This amortized cost applies to each operation, even when there are several types of operations in the sequence.

→ Stack operation:

As an example of aggregate analysis, we analyze stacks that have been augmented with a new operation $MULTIPOP(S, K)$. $MULTIPOP(S, K)$ removes K top objects of stack S , or pops the entire stack if it contains fewer than K objects. The fundamental stack operations $PUSH(x)$ and $POP(S)$ are still present.

$MULTIPOP(S, K)$

```
1 while not STACK-EMPTY(S) and  $K \neq 0$ 
2     do POP(S)
3      $K \leftarrow K - 1$ 
```

Since each fundamental operation runs in $O(1)$ time let us consider the abstract cost of each to be 1. The no. of iterations of the while loop in the pseudocode $MULTIPOP$ is $\min(\text{stack-size}, K)$. Thus, the total cost of $MULTIPOP$ is $\min(\text{stack-size}, K)$ and the actual running time is a linear function of this cost.

Let us analyze a sequence of n $PUSH, POP$ and $MULTIPOP$ operations on an initially empty stack. Since the stack size is at most n , the worst-case cost of a $MULTIPOP$ operation is $O(n)$.

Hence the worst case time of any stack operation is also $O(n)$, and a sequence of n operations cost $O(n^2)$. Although this analysis is correct the

$O(n^2)$ result obtained by considering the worst-case cost of each operation individually is not tight.

Aggregate analysis obtains a better upper bound since it considers the entire sequence of n operations. Each object can be popped at most once for each time it is pushed. Therefore, the no. of times POP can be called on an initially empty stack (including the calls within MULTIPOP) is at most n = the maximum number of PUSH operations. Thus, any sequence of n PUSH, POP, MULTIPOP operations takes a total of $O(n)$ time. Average cost of an operation = $O(n)/n = O(1)$. All three stack operations have an amortized cost of $O(1)$.

→ Incrementing a binary counter:

Consider a K -bit binary counter that counts upward from 0. We use an array $A[0 \dots K-1]$ of bits where length of $A = K$ and the lowest order bit is in $A[0]$.

$$x = \sum_{i=0}^{K-1} A[i] \cdot 2^i$$

Initially x is 0. To increment the value of x we use the following procedure:-

INCREMENT(A)

- 1 $i \leftarrow 0$
- 2 while $i < \text{length}[A]$ and $A[i] = 1$ 3

```

3      do  $A[i] \leftarrow 0$ 
4           $i \leftarrow i+1$ 
5      if  $i < \text{length}[A]$ 
6          then  $A[i] \leftarrow 1$ 

```

At the start of each iteration of the while loop we wish to add a 1 to position i . If $A[i] = 1$ then adding 1 flips the bit to 0 in position i and yields a carry of 1, to be added to position $i+1$ on the next iteration of the loop (maintaining the loop invariant). If the loop terminates because $A[i] = 0$ and $i < \text{length}[A]$ then adding a 1 to $A[i]$ would flip the 0 to 1.

The cost of each increment call is linear in the no. of bits flips. If we perform the worst case analysis of each INCREMENT operation singly: a single execution of INCREMENT takes time $\Theta(k)$ in the worst case. Thus, a sequence of n INCREMENT operations on an initially zero counter takes time $\Theta(nk)$ in the worst case.

To tighten the bound we can perform aggregate analysis. Bit $A[0]$ flips every time INCREMENT is called, bit $A[1]$ flips every other time, bit $A[2]$ flips every fourth time.

In general, for $i = 0, 1, 2, \dots, \lfloor \lg n \rfloor$, bit $A[i]$ flips $\lfloor n/2^i \rfloor$ times in a sequence of n increment operations on an initially zero counter. For $i > \lfloor \lg n \rfloor$ bit $A[i]$ never flips at all.

The total no. of flips in the sequence is

thus

$$\sum_{i=0}^{\lfloor \lg n \rfloor} \left\lfloor \frac{n}{2^i} \right\rfloor < n \sum_{i=0}^{\infty} \frac{1}{2^i} = 2n$$

Therefore, the worst-case time for a sequence of n INCREMENT operations on an initially zero counter is $O(n)$. The amortized cost per operation is $O(n)/n = O(1)$.

3) The accounting method:

In the accounting method of amortized analysis, we assign different charges to different operations, with some operations charged more or less than they actually cost. The amount we charge an operation is called its amortized cost. When an operation's amortized cost exceeds its actual cost, the difference is called credit.

Credit can be used later on to help pay for operations whose amortized cost is less than their actual cost.

→ One must choose the amortized costs of operations carefully. If we want an analysis with amortized costs to show that in the worst case the average cost per operation is small, the total amortized cost of a sequence of operations must be an upper bound on the total actual cost of the sequence. And this relationship must hold for all sequences of operations.

If the actual cost of i th operation is c_i and the amortized cost of i th operation is $\hat{c}_i$ we require

$$\sum_{i=1}^n \hat{c}_i \geq \sum_{i=1}^n c_i \quad \text{for all sequences of } n \text{ operations.}$$

The total credit associated with the data structure is then $\sum_{i=1}^n \hat{c}_i - \sum_{i=1}^n c_i$ which must be non-negative at all times.

→ Accounting method analysis of stack operations

The actual costs of the stack operations here:

PUSH 1

POP 1

MULTIPOP $\min(\text{stack-size}, K)$

Suppose we assign the following amortized costs:-

PUSH $\rightarrow 2$

POP $\rightarrow 0$

MULTIPOP $\rightarrow 0$

It can be shown that we can pay for any sequence of stack operations by charging the amortized costs. Suppose we use a dollar bill to represent each unit of cost. We start with an empty stack.

Suppose the analogy between the stack data structure and a stack of plates in a cafeteria. The actual cost of pushing a plate is 1 dollar but we carried 2 dollars with us (= amortized cost of operation). Thus, we are left with a credit of 1 dollar which we put on top of the plate.

The dollar stored on top of a plate is a prepayment for the cost of popping it from the stack. We execute a POP we pay the actual cost using the credit stored on top of a plate (every plate has it). Thus, in the amortized analysis, by charging the PUSH operations a little bit more we needn't charge the POP or MULTIPOP operations.

Since the stack always has a non-negative no. of plates and every plate has a \$1 credit on it, we have ensured that the total credit is always non-negative. Hence, for "any" sequence of or PUSH, POP and MULTIPOP operations, the total amortized cost is always an upper bound on the total actual cost. Since, the total amortized cost is $O(n)$, so is the total actual cost.

→ An accounting method analysis of incrementing a binary counter

The actual cost of an INCREMENT operation depends upon the no. of bits blipped. Suppose we use a single dollar bill to represent the cost of blipping a single bit.

The actual costs of bit blips are as follows:-

$$\text{bit 0 to bit 1} = 1$$

$$\text{bit 1 to bit 0} = 1$$

Suppose we assign the following amortized costs:-

$$\text{bit 0 to bit 1 (set)} = 2$$

$$\text{bit 1 to bit 0 (clear)} = 0$$

Thus, when we set a bit we use 1 dollar (out of the expected 2 dollars charged) to pay for the actual cost of setting the bit, and deposit one dollar as credit to be used when we later flip the bit back to 0.

An INCREMENT operation comprises of few bit clears (for which nothing is charged) and a maximum of one bit set. Thus the amortized cost of an INCREMENT operation is at most 2 dollars. For a sequence of n operations the total amortized cost is $O(n)$. Since, the no. of 1's in the counter is always non-negative, the total credit is always non-negative.

Therefore the amortized cost bounds the actual cost which also is $O(n)$.

4) The potential method:

The potential method represents the prepaid work as "potential energy" or just potential that can be released to pay for future operations. This potential is associated with the data structure as a whole. In contrast, accounting method represents prepaid work as ~~potential energy~~ stored credit stored in the data structure.

→ We start with an initial data structure D_0 on which n operations are performed. We let c_i represent the actual cost of the i th operation, for $i = 1 \dots n$. Let D_i represent the data structure that results after applying the i th operation to data structure D_{i-1} . The potential associated with data structure D_i is $\phi(D_i)$.

Amortized cost of i th operation is defined to be the actual cost of i th operation plus the increase in potential due to the operation. Amortized cost of i th operation is denoted by $\hat{c}_i$.

$$\hat{c}_i = c_i + \phi(D_i) - \phi(D_{i-1})$$

Total amortized cost of n operation is therefore

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n \{ c_i + \phi(D_i) - \phi(D_{i-1}) \}$$

$$= \sum_{i=1}^n c_i + \phi(D_n) - \phi(D_0)$$

If we define the potential function ϕ such that $\phi(D_n) \geq \phi(D_0)$ then the total amortized cost is an upper bound on the total cost.

In case the number of operations is not known, defining $\phi(D_i) \geq \phi(D_0)$ for every i would do.

→ Stack operations:

Suppose we define the potential function ϕ on a stack to be the no. of objects in the stack.

$$\phi(D_0) = 0 \quad \left\{ \text{since, stack is empty initially} \right\}$$

Since, the stack always contains a non-negative no. of objects we have:-

$$\phi(D_i) \geq 0$$

$$\text{or, } \phi(D_i) \geq \phi(D_0)$$

Thus, the total amortized cost of n operations represents an upper bound on the total actual cost.

Suppose the i th operation of a stack is a PUSH operation, then the potential difference is:-

$$\begin{aligned} \phi(D_i) - \phi(D_{i-1}) &= (\text{stack-size} + 1) - \text{stack-size} \\ &= 1 \end{aligned}$$

The amortized cost of this push operation then is: -

$$\hat{c}_i = c_i + \phi(D_i) - \phi(D_{i-1}) = 1 + 1 = 2$$

[since the actual cost $= c_i = 1$]

Now, suppose the i th operation on the stack is MULTIPOP such that $\text{min}(\text{stack-size}, K) = K'$. Then $c_i = K'$ and potential difference is

$$\phi(D_i) - \phi(D_{i-1}) = -K'$$

The amortized cost of a MULTIPOP operation is therefore: -

$$\begin{aligned}\hat{c}_i &= c_i + \phi(D_i) - \phi(D_{i-1}) \\ &= K' - K' = 0.\end{aligned}$$

Similarly, the amortized cost of a POP operation is also 0.

Since all the three operations cost $O(1)$, the total amortized cost of a sequence of n operations is $O(n)$. Since the total amortized cost is an upper bound on the total cost, the total cost is $O(n)$ in the worst case.

→ Incrementing a binary counter:

Let the no. of 1's in the counter after the i th operation be b_i . We define the potential of the counter after the i th increment to be b_i .

Suppose the i th operation INCREMENT. It clears few 1 bits to 0 and sets a maximum of single zero bit.

If the i th operation clears t_i bits the actual cost of the operation is at most $t_i + 1$. i.e.

$$c_i \leq t_i + 1$$

$$\Rightarrow c_i \leq K + 1 \quad [\because t_i \leq K]$$

If the i th operation clears all bits then $b_i = 0$ and $b_{i-1} = K$. Otherwise

$$b_i = b_{i-1} - t_i + 1$$

In either case, $b_i \leq b_{i-1} - t_i + 1$

The potential difference therefore is:-

$$\begin{aligned} \phi(D_i) - \phi(D_{i-1}) &= b_i - b_{i-1} \\ &\leq (b_{i-1} - t_i + 1) - b_{i-1} \\ &= 1 - t_i \end{aligned}$$

Amortized cost of an INCREMENT operation is therefore:-

$$\begin{aligned} \hat{c}_i &= c_i + \phi(D_i) - \phi(D_{i-1}) \\ &\leq t_i + 1 - (1 - t_i) \\ &= 2 \quad \Rightarrow \hat{c}_i \leq 2 \end{aligned}$$

If the counter starts at 0 then $\phi(D_0) = b_0 = 0$.

$\phi(D_i) \geq 0$ since there are always a non-negative no. of 1 bits. Hence the total amortized cost forms an upper bound on the total actual cost. Since each INCREMENT has an amortized cost of $O(1)$,

a sequence of n INCREMENTS has an amortized cost of $O(n)$. Hence, the total cost is $O(n)$ in the worst case.

→ The potential method gives us a way to analyze the counter even when it does not start at 0.

No. of 1's initially = b_0

No. of 1's after n INCREMENTS = b_n $\left\{ 0 \leq b_0, b_n \leq K \right\}$

$$\sum_i \hat{c}_i = \sum_i c_i + \phi(D_n) - \phi(D_0)$$

$$\Rightarrow \sum_i c_i = \sum_i \hat{c}_i + \phi(D_0) - \phi(D_n)$$

$$= \sum_i c_i + b_0 - b_n$$

$$\leq \sum_{i=1}^n 2 + b_0 - b_n$$

$$= 2n + b_0 - b_n$$

$$= 2n - (b_n - b_0)$$

$$\{ b_n - b_0 \leq K \}$$

$$\Rightarrow \sum_i c_i \leq 2n - (b_n - b_0)$$

This proves that if we execute at least $n = \Omega(K)$ INCREMENT operations the total actual cost is $O(n)$, no matter what initial value the counter contains.